

Aprendizaje por Refuerzo

Grado en Ingeniería Matemática e Inteligencia Artificial (IMAT) – 4º A
Universidad Pontificia Comillas, ICAI

Práctica 1: Ecuaciones de Bellman, Policy y Value Iteration

Informe de laboratorio

Jorge Carnicero
Jorge González
Andrés Gil

Fecha de entrega:
25 de septiembre de 2026

Índice

1. Objetivo y entorno	2
2. Trabajo previo	3
2.1. Ecuaciones de Bellman con la política equiprobable	3
2.2. Rango de r_s para llegar al objetivo lo antes posible	4
2.3. Unicidad de la política óptima	5
3. Policy Iteration	5
3.1. Implementación	5
3.2. Resultados	7
3.3. Influencia de γ y θ	8
4. Value Iteration	10
4.1. Implementación	10
4.2. Resultados	11
4.3. Experimentos: Comparación entre Policy Iteration y Value Iteration . . .	12
5. Experimento adicional: influencia de r_s	14
6. MDP estocástico	15
7. Conclusiones	17
8. Uso de la IA	17

1. Objetivo y entorno

En esta práctica resolvemos un *grid world* inspirado en Flappy Bird con los dos métodos de programación dinámica vistos en clase: Policy Iteration y Value Iteration. Antes de programar nada, planteamos a mano las ecuaciones de Bellman de algunos estados y estudiamos cómo influye la recompensa por paso r_s en la política óptima (trabajo previo).

El entorno (Figura 1) tiene 20 estados no terminales, s_0 a s_{19} , y un único estado terminal T al que se llega chocando con un obstáculo ($r_{obs} = -5$) o alcanzando la celda verde ($r_{goal} = +5$). Cualquier otro movimiento da r_s . Hay dos acciones: arriba-derecha ($\nearrow$, acción 0 en el código) y abajo-derecha ($\searrow$, acción 1). Desde la última columna, cualquier acción hace caer una celda hacia abajo. Salvo que se diga lo contrario, $\gamma = 0.9$ y $\theta = 10^{-3}$.

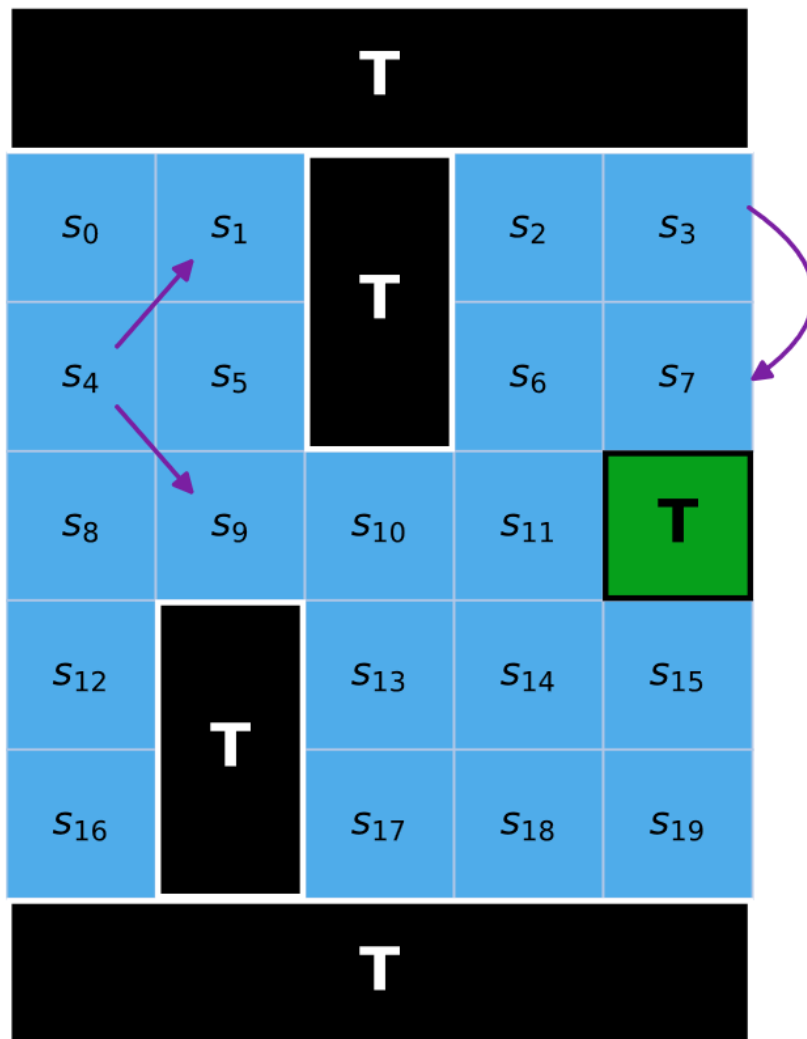


Figura 1: Tablero del entorno. Todas las celdas marcadas con T son el mismo estado terminal.

Aquí adjuntamos un link a nuestro repositorio de github, donde está todo el trabajo realizado:

[reinforcement-learning-lab](#)

2. Trabajo previo

2.1. Ecuaciones de Bellman con la política equiprobable

Pregunta: Determinar, en función de r_s y suponiendo la política estocástica equiprobable en todos los estados, las ecuaciones de Bellman de la función de valor de los estados s_6 , s_{10} y s_5 . Se recomienda resolverlas en ese orden.

Con la política equiprobable, $\pi(\nearrow | s) = \pi(\searrow | s) = 0.5$ en todos los estados. Como el entorno es determinista, la ecuación de Bellman de evaluación queda

$$v_\pi(s) = 0.5 [r_1 + \gamma v_\pi(s'_1)] + 0.5 [r_2 + \gamma v_\pi(s'_2)],$$

donde (s'_1, r_1) y (s'_2, r_2) son el estado siguiente y la recompensa de cada acción. Usamos $\gamma = 0.9$ y $v_\pi(T) = 0$. La Tabla 1 recoge las transiciones que hacen falta.

Estado	Acción $\nearrow$	Acción $\searrow$
s_6	s_3, r_s	T (objetivo), +5
s_3	s_7, r_s (cae)	s_7, r_s (cae)
s_7	T (objetivo), +5	T (objetivo), +5
s_{10}	s_6, r_s	s_{14}, r_s
s_{14}	T (objetivo), +5	s_{19}, r_s
s_{19}	T (obstáculo), -5	T (obstáculo), -5
s_5	T (obstáculo), -5	s_{10}, r_s

Tabla 1: Transiciones de los estados que intervienen en las ecuaciones.

Estado s_6 . Primero necesitamos $v_\pi(s_3)$, que a su vez depende de s_7 . En s_3 y s_7 las dos acciones hacen lo mismo, así que la política da igual:

$$\begin{aligned} v_\pi(s_7) &= 5 + 0.9 \cdot v_\pi(T) = 5, \\ v_\pi(s_3) &= r_s + 0.9 \cdot 5 = r_s + 4.5, \\ v_\pi(s_6) &= 0.5 [r_s + 0.9 v_\pi(s_3)] + 0.5 [5 + 0.9 v_\pi(T)] = 0.5 r_s + 0.45 (r_s + 4.5) + 2.5, \\ \boxed{v_\pi(s_6) &= 0.95 r_s + 4.525} \end{aligned}$$

Estado s_{10} . Depende de s_6 (ya calculado) y de s_{14} , que a su vez depende de s_{19} :

$$\begin{aligned} v_\pi(s_{19}) &= -5, \\ v_\pi(s_{14}) &= 0.5 [5 + 0] + 0.5 [r_s + 0.9 \cdot (-5)] = 0.5 r_s + 0.25, \\ v_\pi(s_{10}) &= 0.5 [r_s + 0.9 v_\pi(s_6)] + 0.5 [r_s + 0.9 v_\pi(s_{14})] \\ &= 0.5 r_s + 0.45 (0.95 r_s + 4.525) + 0.5 r_s + 0.45 (0.5 r_s + 0.25), \\ \boxed{v_\pi(s_{10}) &= 1.6525 r_s + 2.14875} \end{aligned}$$

Estado s_5 . Una acción choca contra el obstáculo central y la otra lleva a s_{10} :

$$\begin{aligned} v_\pi(s_5) &= 0.5 \left[-5 + 0.9 v_\pi(T) \right] + 0.5 \left[r_s + 0.9 v_\pi(s_{10}) \right] \\ &= -2.5 + 0.5 r_s + 0.45 (1.6525 r_s + 2.14875), \end{aligned}$$

$$v_\pi(s_5) = 1.243625 r_s - 1.5330625$$

El orden recomendado tiene sentido: s_6 está a un paso del objetivo, s_{10} usa $v_\pi(s_6)$ y s_5 usa $v_\pi(s_{10})$. Resolviéndolas así no hace falta plantear un sistema de ecuaciones.

2.2. Rango de r_s para llegar al objetivo lo antes posible

Pregunta: Con los valores dados de $r_{obs} = -5$ y $r_{goal} = +5$, y tomando solo para este apartado un factor de descuento $\gamma = 1$, determinar el rango de valores de r_s para el que la política óptima lleva al agente a la celda objetivo lo antes posible desde todo estado desde el que el objetivo sea alcanzable. Justificar los dos extremos del rango: qué hace el agente cuando r_s queda por encima del límite superior y qué hace cuando queda por debajo del inferior.

Primero vemos desde qué estados se puede alcanzar el objetivo. Son todos menos s_1 , s_{15} , s_{16} , s_{18} y s_{19} : desde ellos cualquier camino termina en un obstáculo. En el resto, buscamos que la política óptima elija siempre el camino más corto al objetivo. Con $\gamma = 1$, el retorno de un camino de k pasos intermedios que acaba en el objetivo es $k r_s + 5$.

Límite superior. En s_6 el agente puede ir directo al objetivo ($\searrow$) o dar un rodeo por $s_3 \rightarrow s_7 \rightarrow T$ ($\nearrow$). Prefiere el rodeo si

$$r_s + v_*(s_3) \geq 5 \implies r_s + (r_s + 5) \geq 5 \implies 2r_s \geq 0 \implies r_s \geq 0.$$

Por tanto necesitamos $r_s < 0$. Si r_s queda por encima de este límite, cada paso suma recompensa y al agente le compensa alargar el episodio: elige los caminos más largos que acaben en el objetivo en vez de los más cortos. Con $r_s = 0$ exactamente hay empate entre ir directo y dar el rodeo.

Límite inferior. Ahora el riesgo es que r_s sea tan negativo que el agente prefiera chocar cuanto antes con un obstáculo en vez de intentar llegar a la celda objetivo. Empezamos por ejemplo viendo qué pasa en s_5 , donde podemos chocar directamente ($\nearrow$) o seguir $s_5 \rightarrow s_{10} \rightarrow s_6$ o $s_{14} \rightarrow T$:

$$-5 \geq r_s + v_*(s_{10}) = r_s + (r_s + 5) \implies -10 \geq 2r_s \implies r_s \leq -5.$$

Pero el caso más exigente es el estado más alejado del objetivo que tiene un choque inmediato disponible, porque es el que acumula más pasos con r_s . Es s_{12} : puede chocar directamente ($\searrow$) o recorrer $s_{12} \rightarrow s_9 \rightarrow s_{13} \rightarrow s_{11} \rightarrow s_7 \rightarrow T$ (4 pasos intermedios):

$$-5 \geq 4r_s + 5 \implies -10 \geq 4r_s \implies r_s \leq -2.5.$$

Así que necesitamos $r_s > -2.5$. Si r_s queda por debajo, el agente prefiere chocar con un obstáculo para cortar el episodio (primero desde s_{12} y, cuanto más negativo sea r_s , desde más estados). Con $r_s = -2.5$ exactamente, en s_{12} chocar y llegar al objetivo valen lo mismo.

Resultado:

$$r_s \in (-2.5, 0)$$

El intervalo es abierto porque en los dos extremos aparecen empates, y entonces hay políticas óptimas que no llevan al objetivo lo antes posible. En la Sección 5 comprobamos este comportamiento con los algoritmos.

2.3. Unicidad de la política óptima

Pregunta: ¿Existe una única política óptima en este entorno? Justificar la respuesta.

No. La función de valor óptima v_* es única, pero hay varios estados en los que las dos acciones dan exactamente el mismo retorno, y en esos estados cualquier elección es óptima. Por ejemplo, desde s_{10} ir por $s_{10} \rightarrow s_6 \rightarrow T$ da el mismo retorno que ir por $s_{10} \rightarrow s_{14} \rightarrow T$: los dos caminos tienen la misma longitud y acaban en el objetivo. Además, en la última columna (s_3, s_7, s_{15}, s_{19}) las dos acciones hacen lo mismo, y en s_1 y s_{16} las dos acciones chocan contra un obstáculo. Combinando estas elecciones salen muchas políticas óptimas distintas.

En nuestro código, cuando hay empate, tanto `np.argmax` como `max(..., key=...)` se quedan con la primera acción ($\nearrow$). Por eso en s_{10} la política obtenida va hacia s_6 .

3. Policy Iteration

En esta sección y en las siguientes usamos $r_s = -1$.

3.1. Implementación

Seguimos el orden del enunciado: evaluación, mejora e iteración. Solo mostramos el código que escribimos dentro de los bloques **START CODE HERE / END CODE HERE**. El modelo del entorno viene como `P[s][a] = [(prob, next_state, reward, terminal), ...]`.

Evaluación de la política. Recorremos todos los estados y aplicamos la ecuación de Bellman con la acción que marca la política. Actualizamos V en el mismo vector (sin copiar), y paramos cuando el mayor cambio de un barrido completo es menor que `tol`.

Código 3.1: Evaluación iterativa de la política: `policy_evaluation`

```
while True:
    delta = 0.0

    for state in range(nS):
        old_value = V[state]
        action = int(policy[state])
        # P[s][a] -> [(prob, next_state, reward, terminal), ...]
```

```

    V[state] = sum(
        probability * (reward + gamma * V[next_state]) for
        probability, next_state, reward, terminal in P[state][action]
    )

    delta = max(delta, abs(old_value - V[state]))

if delta < tol:
    break

```

Mejora de la política. Para cada estado calculamos $Q(s, a)$ de las dos acciones con la V evaluada y nos quedamos con la mejor.

Código 3.2: Mejora greedy de la política: policy_improvement

```

# para cada estado, vemos qué acción nos da mayor q(s,a)
for state in range(nS):
    action_values = np.zeros(nA)

    for action in range(nA):
        # en el caso determinista el sumatorio tiene un solo término
        action_values[action] = sum(
            probability*(reward + gamma*value_from_policy[next_state])
            for probability, next_state, reward, terminal
            in P[state][action]
        )

    # la acción con mayor valor es la que entra en la nueva política
    new_policy[state] = int(np.argmax(action_values))

return new_policy

```

Iteración. Alternamos evaluar y mejorar hasta que la política deja de cambiar.

Código 3.3: Bucle de Policy Iteration: policy_iteration

```

while True:
    V = policy_evaluation(P, nS, nA, policy, gamma, tol)
    new_policy = policy_improvement(P, nS, nA, V, gamma)

    converged = np.array_equal(policy, new_policy)
    policy = new_policy

    if converged:
        break

```

3.2. Resultados

Comando 3.4: Ejecución de Policy Iteration en el entorno determinista

```
python main.py --env_behavior deterministic --algorithm policy_iteration
```

```
Entorno: deterministic | gamma=0.9 | tol=0.001
Recompensas: r_s=-1.0 | r_obs=-5.0 | r_goal=5.0
Estados: 20 visitables (s0-s19) + T = 21

-----
POLICY ITERATION - convergencia en 0.036953 s
-----

V*(s) sobre la rejilla:
  T      T      T      T      T
s0      s1      T      s2      s3
0.94    -5.00          3.50    3.50
s4      s5          T      s6      s7
-0.16    2.15          5.00    5.00
s8      s9      s10     s11     T
0.94    0.94    3.50    3.50
s12     T      s13     s14     s15
-0.16          2.15    5.00    -5.50
s16     T      s17     s18     s19
-5.00          3.50    -5.00    -5.00
T      T      T      T      T

V*(T) = 0.00

pi*(s) sobre la rejilla ( / = arriba-derecha, \ = abajo-derecha ):
  T      T      T      T      T
s0      s1      T      s2      s3
\      /          \      /
s4      s5      T      s6      s7
\      \          \      /
s8      s9      s10     s11     T
/      \          /      /
s12     T      s13     s14     s15
/      /          /      /
s16     T      s17     s18     s19
/      /          \      /
T      T      T      T      T

Vector completo (s0 ... s19):
V* = [ 0.935 -5.   3.5  3.5 -0.158 2.15  5.   5.   0.935 0.935
3.5  3.5 -0.158 2.15  5.  -5.5 -5.   3.5 -5.  -5. ]
pi* = [1 0 1 0 1 1 1 0 0 1 0 0 0 0 0 0 0 0 1 0]

Valor de los estados iniciales:
V*(s0)= 0.935 V*(s4)= -0.158 V*(s8)= 0.935 V*(s12)= -0.158

Episodio desde s0: s0 -> s5 -> s10 -> s6 -> T
Recompensa acumulada (sin descuento): 2.00
```

Figura 2: Salida de Policy Iteration con $\gamma = 0.9$, $\theta = 10^{-3}$ y $r_s = -1$: V_* y π_* sobre la rejilla (/ = $\nearrow$, \ = $\searrow$).

Revisando estado a estado, vemos que la política obtenida es óptima: en cada estado desde el que se puede llegar al objetivo, el agente toma la acción que le lleva por el camino más corto. Por ejemplo, desde s_0 el episodio es $s_0 \rightarrow s_5 \rightarrow s_{10} \rightarrow s_6 \rightarrow T$, con recompensa acumulada $-1 - 1 - 1 + 5 = 2$. Su valor descontado cuadra con el de la figura:

$$v_*(s_0) = -1 - 0.9 - 0.81 + 0.9^3 \cdot 5 = 0.935.$$

En los estados desde los que el objetivo no es alcanzable, el agente minimiza la pérdida. El caso más claro es s_{18} : si sube a s_{15} acabará cayendo por s_{19} al obstáculo inferior con valor $-1 - 0.9 + 0.9^2 \cdot (-5) = -5.95$, así que prefiere chocar ya directamente y quedarse tan solo en un valor de -5 . En s_1 y s_{16} no hay elección posible: las dos acciones chocan.

3.3. Influencia de γ y θ

Pregunta: Analizar el proceso de aprendizaje del agente y realizar un análisis sobre cómo se ve afectado el aprendizaje del agente y la convergencia del mismo para distintos valores de los hiperparámetros γ y umbral de convergencia θ .

Probaremos todas las combinaciones de $\gamma \in \{0.5, 0.9, 0.99\}$ y $\theta \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}\}$ en el entorno determinista con Policy Iteration (12 ejecuciones).

γ	$v_*(s_0)$	$v_*(s_4)$	PI: iter. externas	PI: barridos ev.
0.5	-1.1250	-1.5625	3	12
0.9	0.9350	-0.1585	3	11
0.99	1.8814	0.8626	3	11

Tabla 2: Resultados por valor de γ . Son idénticos para las cuatro tolerancias probadas.

Efecto de θ . La tolerancia no influye: ni el número de barridos ni los valores finales cambian entre $\theta = 10^{-1}$ y $\theta = 10^{-4}$, como muestra la Figura 3.

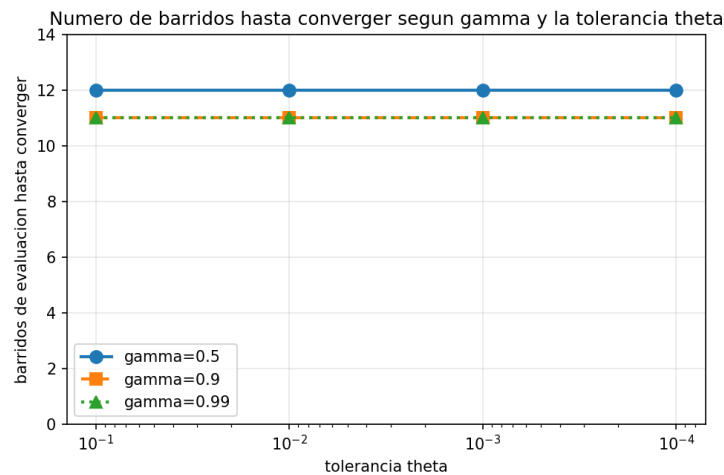


Figura 3: Número de barridos hasta converger según γ y la tolerancia θ .

Además, la diferencia máxima en V respecto a la ejecución con $\theta = 10^{-4}$ es exactamente 0 en los 12 casos, muy por debajo del límite permitido por cada tolerancia (Figura 4). El motivo es que el agente siempre avanza una columna a la derecha (o baja una celda en la última), así que no hay ciclos y cualquier episodio termina en pocos pasos: en unos pocos barridos los valores ya son exactos, y el último barrido no cambia nada, con lo que el Δ que detiene el bucle es 0, por debajo de cualquier θ probado. En un entorno con ciclos los valores se acercarían poco a poco a su límite, y ahí sí que un θ más pequeño daría más precisión a cambio de más barridos.

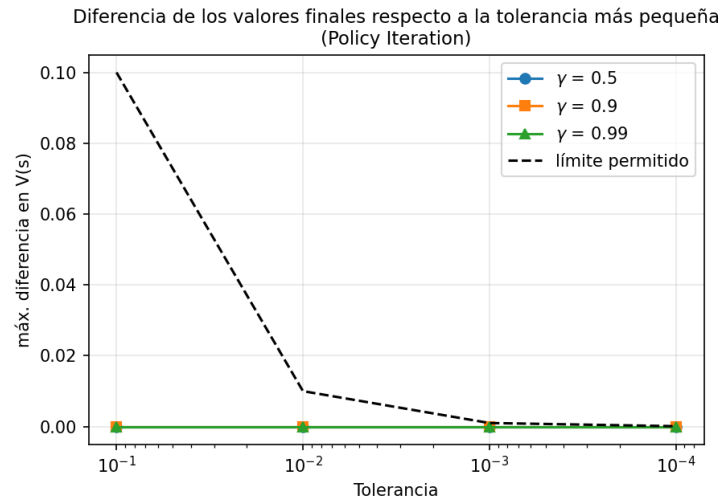


Figura 4: Diferencia máxima de V (Policy Iteration) respecto a la ejecución con $\theta = 10^{-4}$, junto al límite permitido por cada tolerancia.

Efecto de γ . γ sí cambia los valores: al pesar más la recompensa final frente al -1 de cada paso, $v_*(s_0)$ pasa de -1.125 ($\gamma = 0.5$) a 1.881 ($\gamma = 0.99$) (Tabla 2). Con $\gamma = 0.5$ incluso cambia la política en s_{18} : el agente prefiere subir a s_{15} y chocar más tarde ($-1 + 0.5 \cdot (-3.5) = -2.75 > -5$), un comportamiento más cortoplacista. La convergencia apenas se ve afectada: 12 barridos con $\gamma = 0.5$ frente a 11 con los otros dos, siempre en 3 iteraciones externas.

Proceso de aprendizaje. Por último, la Figura 5 muestra la evolución de V en los estados iniciales por iteración externa de Policy Iteration. Con la política inicial ($\nearrow$ en todos los estados) casi todos chocan ($v \approx -5$); en la segunda iteración mejoran algunos estados, y en la tercera todos alcanzan su valor óptimo y la política deja de cambiar.

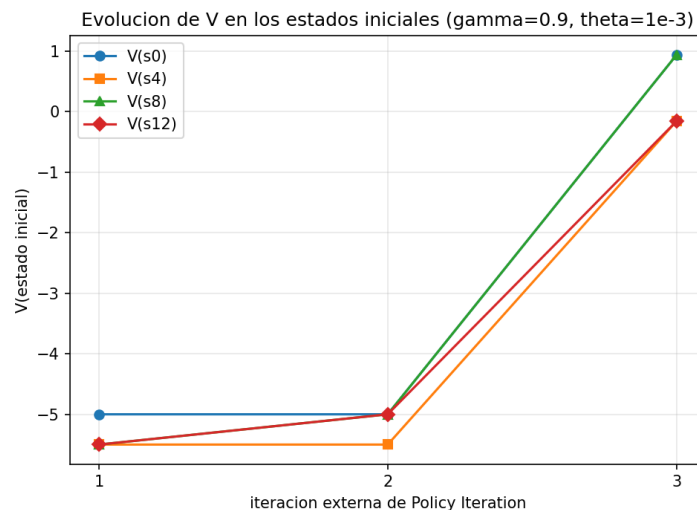


Figura 5: Evolución de V en los estados iniciales ($\gamma = 0.9$, $\theta = 10^{-3}$), por iteración externa de Policy Iteration.

4. Value Iteration

4.1. Implementación

En Value Iteration no hay una política que evaluar: en cada barrido aplicamos directamente la ecuación de optimalidad de Bellman, quedándonos con el máximo sobre las dos acciones. Cuando el mayor cambio de un barrido baja de `tol`, sacamos la política greedy respecto a la V final.

Código 4.1: Value Iteration: `value_iteration`

```
while True:

    delta = 0

    for state in range(nS):

        old_value = V[state]

        V[state] = max(
            sum(probability * (reward + gamma * V[next_state])
                for probability, next_state, reward, terminal
                in P[state][action]
            )
            for action in range(nA)
        )

        delta = max(delta, abs(old_value - V[state]))

    if delta < tol:
        break

policy = np.array([
    max(range(nA), key=lambda action:

        sum(probability * (reward + gamma * V[next_state])
            for probability, next_state, reward, terminal
            in P[state][action]))

    for state in range(nS)
])
```

4.2. Resultados

Comando 4.2: Ejecución de Value Iteration en el entorno determinista

```
python main.py --env_behavior deterministic --algorithm value_iteration
```

```
Entorno: deterministic | gamma=0.9 | tol=0.001
Recompensas: r_s=-1.0 | r_obs=-5.0 | r_goal=5.0
Estados: 20 visitables (s0-s19) + T = 21

-----
VALUE ITERATION - convergencia en 0.001000 s
-----
V*(s) sobre la rejilla:
  T      T      T      T      T
s0      s1      T      s2      s3
0.94    -5.00      3.50    3.50
s4      s5      T      s6      s7
-0.16    2.15      5.00    5.00
s8      s9      s10     s11     T
0.94    0.94    3.50    3.50
s12     T      s13     s14     s15
-0.16    2.15    5.00    -5.50
s16     T      s17     s18     s19
-5.00    3.50    -5.00    -5.00
T      T      T      T      T

V*(T) = 0.00

pi*(s) sobre la rejilla ( / = arriba-derecha, \ = abajo-derecha ):
  T      T      T      T      T
s0      s1      T      s2      s3
\      /      \      /
s4      s5      T      s6      s7
\      \      \      /
s8      s9      s10     s11     T
/      \      /      /
s12     T      s13     s14     s15
/      /      /      /
s16     T      s17     s18     s19
/      /      \      /
T      T      T      T      T

Vector completo (s0 ... s19):
V* = [ 0.935 -5.   3.5   3.5  -0.158 2.15  5.   5.   0.935 0.935
3.5   3.5  -0.158 2.15  5.   -5.5  -5.   3.5  -5.   -5. ]
pi* = [1 0 1 0 1 1 1 0 0 1 0 0 0 0 0 0 0 0 1 0]

Valor de los estados iniciales:
V*(s0)= 0.935 V*(s4)= -0.158 V*(s8)= 0.935 V*(s12)= -0.158

Episodio desde s0: s0 -> s5 -> s10 -> s6 -> T
Recompensa acumulada (sin descuento): 2.00
```

Figura 6: Salida de Value Iteration con $\gamma = 0.9$, $\theta = 10^{-3}$ y $r_s = -1$.

Si nos fijamos, vemos que obtenemos exactamente los mismos valores y la misma política que habíamos obtenido antes con Policy Iteration (Figura 2), lo que nos sirve también como comprobación de que las dos implementaciones son correctas.

Por defecto, el episodio que se visualiza al ejecutar comienza en el estado s_0 , por lo que probamos desde los otros estados iniciales:

Comando 4.3: Ejecución desde los demás estados iniciales

```
python main.py --env_behavior deterministic --algorithm value_iteration
  ↪ --start_state 4
python main.py --env_behavior deterministic --algorithm value_iteration
  ↪ --start_state 8
python main.py --env_behavior deterministic --algorithm value_iteration
  ↪ --start_state 12
```

```

Episodio desde s4: s4 -> s9 -> s13 -> s11 -> s7 -> T
Recompensa acumulada (sin descuento): 1.00

Episodio desde s8: s8 -> s5 -> s10 -> s6 -> T
Recompensa acumulada (sin descuento): 2.00

Episodio desde s12: s12 -> s9 -> s13 -> s11 -> s7 -> T
Recompensa acumulada (sin descuento): 1.00

```

Figura 7: Episodios seguidos por el agente desde s_4 , s_8 y s_{12} .

Desde s_4 y s_{12} el camino más corto tiene 4 pasos intermedios ($\rightarrow s_9 \rightarrow s_{13} \rightarrow s_{11} \rightarrow s_7 \rightarrow T$) y la recompensa acumulada es $-4 + 5 = 1$. Desde s_8 el agente sigue el mismo camino que desde s_0 a partir de s_5 y acumula 2.

4.3. Experimentos: Comparación entre Policy Iteration y Value Iteration

Pregunta: Comparar el desempeño del agente cuando aprende mediante Policy Iteration y cuando lo hace con Value Iteration. ¿Se llega siempre a la misma solución?

En los experimentos realizados sí. Para todos los valores de γ y θ estudiados en el entorno determinista, Policy Iteration y Value Iteration obtienen los mismos valores. En la Figura 8 se observa cómo las curvas de ambos métodos se superponen para los estados iniciales.

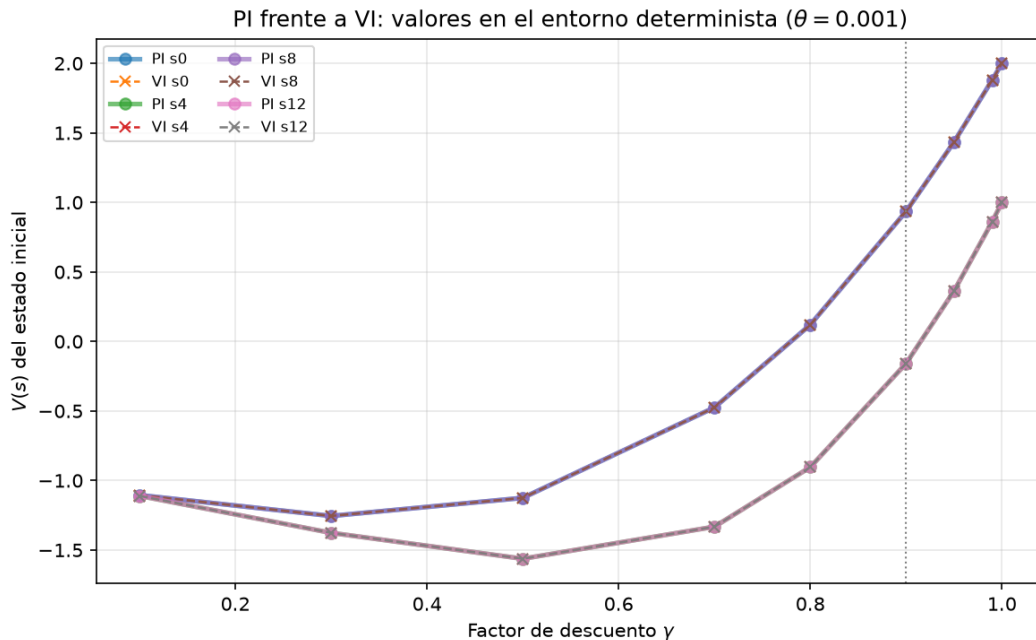


Figura 8: Comparación de los valores obtenidos por Policy Iteration y Value Iteration al variar γ , manteniendo $\theta = 10^{-3}$.

Pregunta: ¿Son las políticas óptimas iguales?

En nuestros experimentos sí. Pero no tiene por qué ser así en general: como vimos en el trabajo previo, hay estados con empate entre las dos acciones (por ejemplo s_{10}) y cada método podría elegir una distinta sin dejar de ser óptima. En nuestro caso coinciden porque las dos implementaciones resuelven los empates igual, quedándose con la primera acción.

Pregunta: ¿Convergen en tiempos similares?

En nuestras ejecuciones, ambos algoritmos tardan menos de un milisegundo, aunque Value Iteration necesita aproximadamente la mitad de tiempo. Con $\gamma = 0.9$ y $\theta = 10^{-3}$, Policy Iteration realiza 3 iteraciones externas y 11 barridos de evaluación, mientras que Value Iteration converge en 4 barridos.

Estos barridos no representan exactamente el mismo trabajo computacional. Por ello, también contamos las evaluaciones de $Q(s, a)$ realizadas: Policy Iteration necesita 357 y Value Iteration 168. Esto concuerda con la estructura de los algoritmos: Policy Iteration evalúa cada política intermedia hasta cumplir el criterio de convergencia antes de mejorarla, mientras que Value Iteration aplica directamente la ecuación de optimalidad de Bellman en cada barrido.

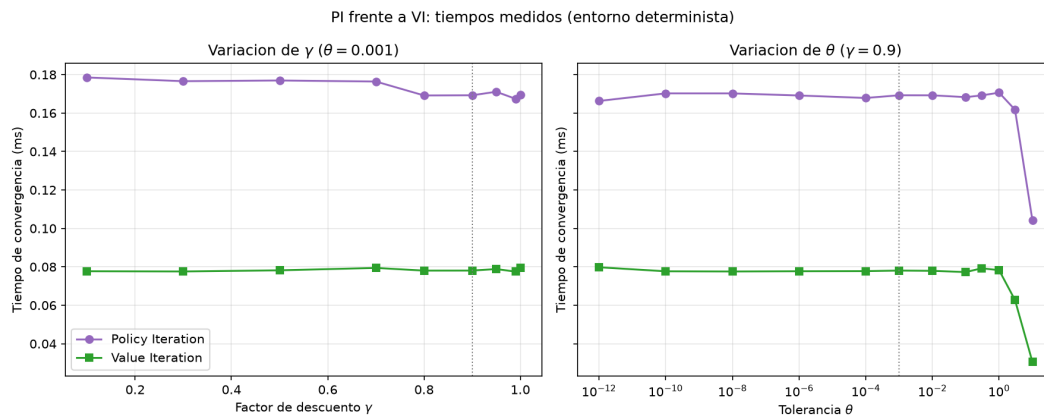


Figura 9: Tiempos medidos de Policy Iteration y Value Iteration al variar γ y θ en el entorno determinista.

Los tiempos dependen de la máquina y se consideran una medida secundaria. Los resultados corresponden a este entorno y a nuestra implementación; no implican que Value Iteration sea siempre más eficiente.

5. Experimento adicional: influencia de r_s

Para comprobar con los algoritmos lo que razonamos en el trabajo previo, ejecutamos Policy y Value Iteration con $r_s \in \{1, 0, -0.5, -1, -1.5, -2, -2.5, -5\}$, dejando $\gamma = 0.9$ y $\theta = 10^{-3}$. Observamos:

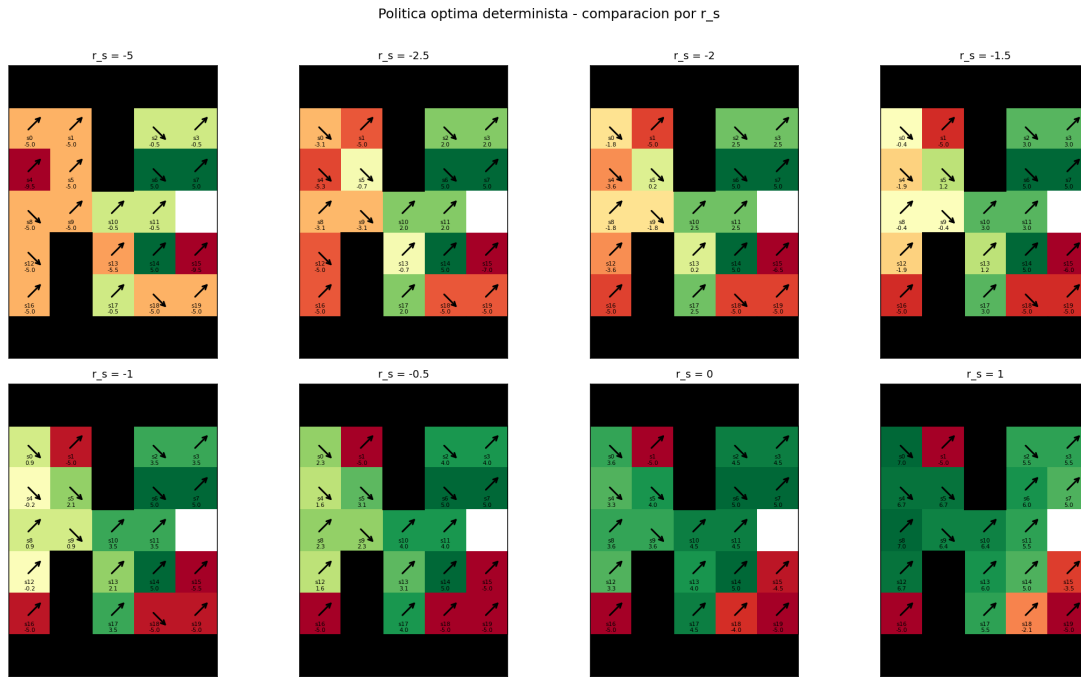


Figura 10: Política óptima (flechas) y V^* (color) para cada valor de r_s .

Los resultados cuadran con el trabajo previo:

- **Por encima del límite superior** ($r_s > 0$). Desde s_0 y s_8 el agente ya no va por el camino corto de 4 pasos, sino que da más steps $\dots \rightarrow s_6 \rightarrow s_3 \rightarrow s_7 \rightarrow T$
- **Dentro del rango** ($-2.5 < r_s \leq 0$). Todos los estados iniciales llegan al objetivo por el camino más corto. La política prácticamente no cambia; solo cambian los valores, que bajan a medida que r_s es más negativo.
- **En el límite inferior** ($r_s = -2.5$). s_{12} ya prefiere chocar en el primer paso. Es el estado que identificamos como el más exigente.
- **Muy por debajo** ($r_s = -5$). Ningún estado inicial llega al objetivo: todos chocan cuanto antes.

6. MDP estocástico

Pregunta: Asumir ahora que las transiciones del entorno pasan a ser estocásticas, de manera que: en todos los estados y pares (estado, arriba-derecha) el agente realiza la acción arriba-derecha con $p_{\text{exito}} = 0.8$ y con $p_{\text{fallo}} = 0.2$ va abajo-derecha. Del mismo modo, para todos los estados y pares (estado, abajo-derecha) el agente realiza la acción abajo-derecha con $p_{\text{exito}} = 0.8$ y con $p_{\text{fallo}} = 0.2$ va arriba-derecha. Analizar el proceso de aprendizaje del agente y compararlo con su desempeño en el entorno determinista.

Comando 6.1: Ejecución en el entorno estocástico

```
python main.py --env_behavior stochastic --algorithm policy_iteration
python main.py --env_behavior stochastic --algorithm value_iteration
```

Los dos algoritmos vuelven a dar entre sí la misma V_* y la misma π_* , lo que nos sirve otra vez de comprobación cruzada. Lo que es resaltable es que la política óptima es exactamente la misma que en el entorno determinista: en cada estado sigue siendo elegida la misma acción. Esto se debe sobre todo a que la probabilidad de éxito asociada a ambas acciones es un valor suficientemente alto, pero si lo fuéramos reduciendo veríamos que en cierto punto empezaría a cambiar la política óptima. Por ejemplo, si la probabilidad de éxito fuera 0.2 y la de fallo fuera 0.8, entonces obtendríamos la política completamente opuesta a la que tenemos ahora.

Por otro lado, lo que sí que cambia y de hecho se reducen, son los valores de los estados. Ahora en la ecuación de Bellman tenemos que promediar los dos sucesores posibles en cada estado para calcular su valor dada una acción:

$$q_*(s, a) = 0.8 [r + \gamma v_*(s'_{\text{acierto}})] + 0.2 [r + \gamma v_*(s'_{\text{fallo}})],$$

así que el riesgo de acabar en el sucesor que no queríamos entra en el valor, degradándolo. Eso empeora ambas dos acciones a la vez y en nuestro caso, no llega a cambiar cuál de las dos es mejor: el **argmax** no se mueve, pero el retorno esperado sí. Veamos algunos ejemplos:

Estado	v_*^{det}	v_*^{sto}	Δv_*	π_*
s_3 (última columna, sin elección real)	3.500	3.500	0.000	$\nearrow = \nearrow$
s_6 (a un paso del objetivo, fallo seguro)	5.000	4.430	0.570	$\searrow = \searrow$
s_{14} (a un paso del objetivo, fallo peligroso)	5.000	2.900	2.100	$\nearrow = \nearrow$
s_9 (fallo directo contra el obstáculo central)	0.935	-2.358	3.293	$\searrow = \searrow$
s_{17} (camino largo por la zona de abajo)	3.500	0.288	3.212	$\nearrow = \nearrow$
s_4 (el más alejado del objetivo)	-0.158	-3.598	3.439	$\searrow = \searrow$

Tabla 3: Valores óptimos en el entorno determinista y en el estocástico ($p_{\text{exito}} = 0.8$, $\gamma = 0.9$), con $\Delta v_* = v_*^{det} - v_*^{sto}$. La política óptima coincide en los 20 estados.

La Tabla 3 nos muestra que la penalización debida a la estocasticidad no es igual para todos los estados. Δv_* es 0 en los estados en los que las dos acciones hacen lo mismo (s_1 , s_3 , s_7 , s_{15} , s_{16} y s_{19}), porque ahí fallar la acción da igual. Y compara s_6 con s_{14} : los dos están a un paso del objetivo, pero desde s_6 el fallo lleva a s_3 , que todavía acaba en el objetivo ($\Delta v_* = 0.57$), mientras que desde s_{14} el fallo lleva a s_{19} , que solo puede chocar ($\Delta v_* = 2.10$). También se ve que el efecto se propaga hacia atrás: s_4 y s_{17} no están pegados a ningún obstáculo, pero pierden más de 3 puntos porque su camino óptimo pasa por más zonas de riesgo.

Esto se observa más claramente en la siguiente imagen:

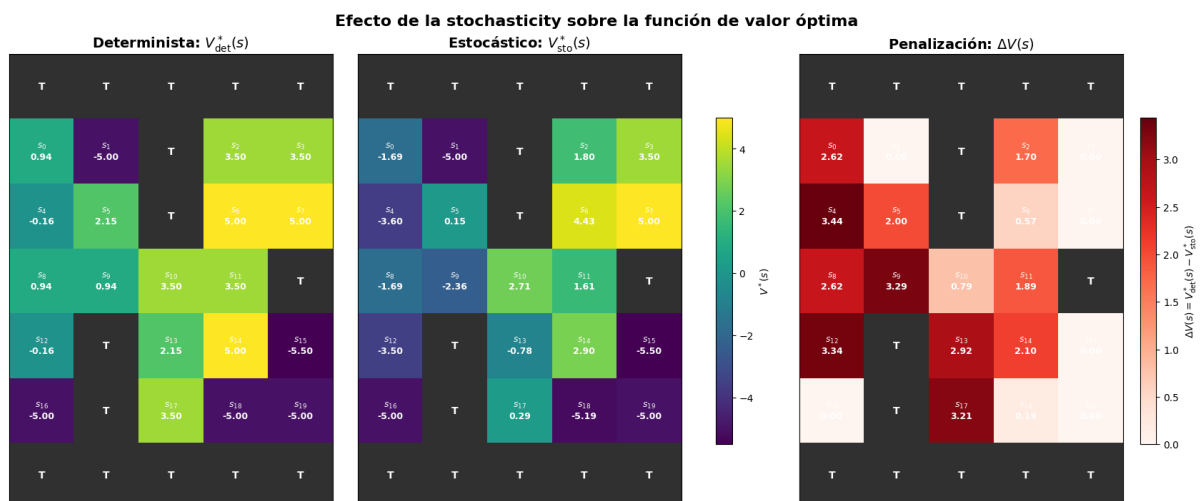


Figura 11: Evolución de la función de valor óptima con estocasticidad

La otra diferencia que vemos al ejecutar los algoritmos en el modo estocástico es que ahora el episodio que se visualiza cambia de una ejecución a otra. Con la misma π_* y saliendo siempre de s_0 recogimos, entre otros, $s_0 \rightarrow s_5 \rightarrow s_{10} \rightarrow s_6 \rightarrow T$ (retorno +2), $s_0 \rightarrow T$ (retorno -5, el fallo en el primer paso lleva al obstáculo de arriba) y $s_0 \rightarrow s_5 \rightarrow s_{10} \rightarrow s_{14} \rightarrow s_{19} \rightarrow T$ (retorno -9). Esto no quiere decir que V_* o π_* sean aleatorios: Policy y Value Iteration usan el modelo $p(s', r | s, a)$ y siempre devuelven lo mismo. Lo aleatorio es la trayectoria que ejecuta el entorno. De hecho $v_*(s_0) = -1.69$ es el retorno esperado desde s_0 , un número que ningún episodio concreto tiene por qué dar, ya que solo es el valor esperado, pero en cada ejecución concreta se obtiene uno u otro.

7. Conclusiones

La función de valor óptima es única, pero la política óptima no. En varios estados las dos acciones dan exactamente el mismo retorno, así que hay muchas políticas óptimas distintas. Que Policy y Value Iteration nos devuelvan la misma política no es una propiedad del MDP, sino de cómo desempatan nuestras implementaciones.

La recompensa por paso decide el comportamiento del agente, no solo sus valores. Con $\gamma = 1$ el agente va al objetivo lo antes posible solo si $r_s \in (-2.5, 0)$. Fuera de ese rango sigue siendo óptimo, pero optimiza otra cosa: si $r_s > 0$ alarga el episodio todo lo que puede, y si $r_s < -2.5$ prefiere chocar cuanto antes contra un obstáculo.

Una política óptima no garantiza llegar al objetivo. Desde $s_1, s_{15}, s_{16}, s_{18}$ y s_{19} el objetivo es inalcanzable, y ahí lo óptimo es minimizar la pérdida: s_{18} choca directamente (-5) en vez de encadenar pasos hasta caer por s_{19} (-5.95) . El agente maximiza retorno esperado, no tasa de éxito.

γ cambia los valores y, en el límite, la política. Al subir γ pesa más el $+5$ del final frente a los -1 del camino. Con $\gamma = 0.5$ el agente se vuelve cortoplacista y en s_{18} prefiere retrasar el choque en vez de asumirlo ya. La tolerancia θ , en cambio, apenas influye: este entorno no tiene ciclos y los episodios son cortos, así que los valores se vuelven exactos en pocos barridos y el criterio de parada se cumple igual con $\theta = 10^{-1}$ que con 10^{-12} .

Policy Iteration y Value Iteration resuelven el mismo problema, con distinto coste. En todas las configuraciones probadas (determinista y estocástica, variando γ, θ y r_s) los dos llegan a la misma V_* y a la misma π_* , lo que nos sirve como validación cruzada de las dos implementaciones. Value Iteration necesita menos trabajo (168 evaluaciones de $Q(s, a)$ frente a 357) porque no evalúa hasta el final políticas intermedias que va a descartar. Es un resultado de este entorno y esta implementación, no una regla general.

La incertidumbre no cambia qué acción es mejor, pero sí cuánto vale. En el entorno estocástico con $p_{\text{exito}} = 0.8$ la política óptima coincide estado a estado con la determinista, mientras que V_* baja en todos los estados donde las dos acciones llevan a sitios distintos. El Δv_* mide precisamente el coste de la incertidumbre, y no depende solo de la distancia al obstáculo sino de si un fallo puede acabar en él. Ese coste se propaga hacia atrás por las ecuaciones de Bellman hasta estados que no tocan ningún obstáculo.

Solución óptima fija no es trayectoria fija. Los dos algoritmos son métodos model-based: usan $p(s', r | s, a)$ y devuelven siempre lo mismo. Lo que cambia entre ejecuciones es el episodio que ejecuta el entorno. Por eso una cosa es el retorno esperado, un valor que ningún episodio concreto tiene por qué dar, y otra cosa es el valor que nos sale en cada episodio concreto.

8. Uso de la IA

- Localizar faltas ortográficas, mejora de la cohesión y fluidez del texto.
- Ayuda a la hora de generación de plots.
- Ayuda en la generación de tablas y formulas en Latex